

What is this assignment about?

You are given a pseudo code program. Your job is to:

1. Draw a **CFG** (only decision points as a graph)
2. Draw an **HFG** (all statements grouped into a graph)
3. Calculate **Statement Coverage** (how many lines run for a given input?)
4. Calculate **Decision Coverage** (how many paths/branches are tested?)

(0) The Pseudo Code (Your Program)

Your roll number ends in **22**. This means all calculations use the number 22. Here is the full program with line numbers:

#	Code	Type	Simple Meaning
1	Read 22	Input	Take 22 as input (your roll number)
2	A = 22 * -1	Calc	A = -22
3	B = 22 * -10	Calc	B = -220
4	C = 22 * 50	Calc	C = 1100
5	X = C + A - B	Calc	X = 1100 + (-22) - (-220) = 1298
6	Y = C + B - A	Calc	Y = 1100 + (-220) - (-22) = 902
7	Z = A + B + C	Calc	Z = (-22) + (-220) + 1100 = 858
8	IF X > Y THEN	DECISION	Is 1298 > 902? YES -> go inside
9	IF X > Z THEN	DECISION	Is 1298 > 858? YES -> go inside
10	I = 0	Action	Set I to 0
11	WHILE I<10 DO	DECISION	Loop while I is less than 10
12	Print A	Action	Print A (-22)
13	END WHILE	DECISION	Go back to check WHILE condition
14	Print B	Action	Print B
15	ELSE	DECISION	If X was NOT > Z

16	Print C	Action	Print C
17	END IF	DECISION	End of inner IF
18	ELSE	DECISION	If X was NOT > Y
19	IF Y > Z THEN	DECISION	Is Y > Z?
20	I = 0	Action	Set I to 0
21	DO	DECISION	Start do-while loop (run body first)
22	Print A	Action	Print A
23	WHILE I < 10	DECISION	Check condition AFTER body runs
24	Print B	Action	Print B
25	ELSE	DECISION	If Y was NOT > Z
26	Print C	Action	Print C
27	END IF	DECISION	End of inner IF
28	END IF	DECISION	End of outer IF

Key observation:

For input 22: X=1298, Y=902, Z=858. So X > Y (TRUE) and X > Z (TRUE). This means the program always takes the TT path (both IFs are true). Lines 15-27 are NEVER executed!

(a) Control Flow Graph (CFG)

Simple Analogy - Think of a road map:

Imagine you are driving. Most roads are straight (no decision). But at traffic signals or junctions, you DECIDE to go left or right. CFG only draws the JUNCTIONS (decision points) - it ignores the straight roads in between.

Step 1: What to KEEP and what to REMOVE?

KEEP in CFG (Decision statements)

- IF ... THEN
- ELSE
- END IF
- WHILE ... DO
- END WHILE
- DO (start of do-while)
- WHILE (end of do-while)

REMOVE from CFG (Non-decision)

- Read / Input
- Assignments (A = ...)
- Print statements
- Calculations (X = C+A-B)
- All lines 1 to 7 in this program

Step 2: What remains after removing non-decisions?

After removing lines 1-7 and other non-decision lines, only these remain:

Line	Statement	Why it's a decision?
8	IF X > Y THEN	Chooses between 2 paths: true or false
9	IF X > Z THEN	Another choice inside the true branch
11	WHILE I < 10 DO	Loop: decides whether to repeat or exit
13	END WHILE	Loop check point - goes back or moves on
15	ELSE	The false side of line 9's IF
17	END IF	Closes the inner IF (line 9)
18	ELSE	The false side of line 8's IF
19	IF Y > Z THEN	Another choice in the false branch
21	DO	Start of do-while (run body first!)
23	WHILE I < 10	Loop check AFTER body - post-test
25	ELSE	False side of line 19's IF
27	END IF	Closes inner IF (line 19)
28	END IF	Closes outer IF (line 8)

Step 3: Drawing Rules (NEVER forget these!)

Rule	What it means
TRUE always goes LEFT	When an IF is true, the arrow goes to the LEFT side
FALSE always goes RIGHT	When an IF is false, the arrow goes to the RIGHT side
END IF is directly below IF	Every IF must have its END IF placed straight below it
WHILE loops back up	WHILE's true arrow goes BACK UP to the WHILE node (loop!)
DO-WHILE: body first	In do-while, the DO box comes before the WHILE check box

Step 4: Loop Types - The KEY difference

PRE-TEST loops (WHILE, FOR)

- Condition checked FIRST
- If false at start: body NEVER runs
- In CFG: WHILE box is at TOP
- True -> goes into body
- After body -> goes BACK to WHILE
- False -> exits loop
- Example: Line 11 WHILE I<10 DO

POST-TEST loops (DO WHILE)

- Body runs FIRST, condition checked after
- Body ALWAYS runs at least once
- In CFG: DO box is at TOP
- Then body runs
- Then WHILE check at BOTTOM
- True -> goes BACK to DO
- Example: Lines 21-23 DO...WHILE I<10

(b) Hybrid Flow Graph (HFG)

Simple Analogy - A more detailed road map:

If CFG is a map with only junctions, HFG is a map with junctions AND road signs. You group all the straight roads between two junctions into ONE labeled segment. Decision points remain as individual nodes, but straight stretches get bundled together.

CFG vs HFG - Simple comparison

	CFG	HFG
What it keeps	Decision statements ONLY	ALL executable statements
What it removes	Non-decisions + non-exec	Only non-executable (#include etc.)
Non-decision exec	Completely DROPPED	Kept and GROUPED together
Result	Few nodes (only junctions)	More nodes (groups + junctions)
Example line 12	Print A - REMOVED	Print A - kept in Group 3

How to create groups?

- Find consecutive non-decision executable statements (assignments, prints, reads)
- Bundle them into ONE group node (circle with a number)
- When you hit a decision statement - stop the group, start a new one later
- Each group gets a number: 1, 2, 3...

Group No.	Line(s)	What's inside	Why grouped together?
Group 1	1 to 7	READ, A=, B=, C=, X=, Y=, Z=	All 7 lines are consecutive non-decision statements
Group 2	10	I = 0	Sits between decision lines 9 and 11

Group 3	12	PRINT A	Inside the WHILE loop body (between lines 11 and 13)
Group 4	14	PRINT B	After END WHILE (line 13), before ELSE (line 15)
Group 5	16	PRINT C	Inside ELSE branch (between lines 15 and 17)
Group 6	20	I = 0	Between decision lines 19 and 21
Group 7	22	PRINT A	Inside DO-WHILE body (between lines 21 and 23)
Group 8	24	PRINT B	After DO-WHILE ends (after line 23)
Group 9	26	PRINT C	Inside ELSE branch (between lines 25 and 27)

Remember:

In your HFG diagram, wherever you see a group of executable lines, you just write the group number (1, 2, 3...) inside a circle/oval. Decision statements keep their own individual node.

(c) Statement Coverage

Simple Analogy - Checking which rooms you visited:

Imagine a house has 28 rooms. You walk through the house with a specific route (test input). Statement Coverage asks: out of 28 rooms, how many did you actually enter? The higher the percentage, the more of the code you tested.

Formula:

$$\text{Statement Coverage} = (\text{Lines that executed} / \text{Total lines}) \times 100$$

Total lines in this program = **28**

How to calculate? Do a Dry Run!

A **dry run** means you trace through the code manually with a specific input and note down every line that actually executes. Like walking through the house yourself.

Dry Run for Input = 22:

Line	Statement	Execute s?	Reason
------	-----------	------------	--------

1-7	READ, A, B, C, X, Y, Z	YES	Always runs at the start
8	IF X > Y	YES	X=1298 > Y=902 -> TRUE, enter IF
9	IF X > Z	YES	X=1298 > Z=858 -> TRUE, enter IF
10	I = 0	YES	Inside true branch
11	WHILE I<10	YES	I=0 < 10 -> loop runs
12	Print A	YES	Inside loop body
13	END WHILE	YES	Loop check point
14	Print B	YES	After loop exits
15	ELSE	NO	We took the TRUE path, ELSE is skipped
16	Print C	NO	Inside ELSE - skipped
17	END IF	NO	Skipped
18	ELSE	NO	Outer IF was true, so ELSE is skipped
19-27	All inside ELSE	NO	All skipped because outer IF was true
28	END IF	YES	Always runs at end

Result for Input 22:

Lines executed: 1,2,3,4,5,6,7,8,9,10,11,12,13,14,28 = 16 lines

Statement Coverage = $16 / 28 \times 100 = 57.14\%$

Dry Run for Input = -22:

With -22: A=22, B=220, C=-1100. X = -1100+22-220 = -1298, Y=-1100+220-22=-902, Z=22+220-1100=-858.

X(-1298) > Y(-902)? NO -> outer IF is FALSE -> go to ELSE (line 18).

Y(-902) > Z(-858)? NO -> inner IF is FALSE -> go to ELSE (line 25).

Path taken: **FF (False-False)**

Result for Input -22:

Lines executed: 1,2,3,4,5,6,7,8,18,19,25,26,27,28 = 14 lines

Statement Coverage = $14 / 28 \times 100 = 50\%$

Summary Table:

Input	Path	Lines that run	Count	Coverage
22	TT (both IFs true)	1-14, 28	16 lines	57.14%

-22	FF (both IFs false)	1-8, 18-19, 25-28	14 lines	50%
-----	---------------------	-------------------	----------	-----

(d) Decision Coverage

Simple Analogy - Testing all routes on a map:

Your program has branch points (IFs). Each IF can go TRUE or FALSE. Decision coverage asks: out of ALL possible routes through the map, how many did you actually test? If there are 4 routes and you tested 1, coverage = 25%.

Formula:

$$\text{Decision Coverage} = (\text{Paths executed} / \text{Total paths}) \times 100$$

How many paths exist?

This program has **2 nested IFs**. Each IF has 2 outcomes (T or F). So total paths = $2 \times 2 = 4$ paths.

Path Name	Outer IF (line 8) X > Y	Inner IF (line 9 or 19)	What happens	Which input triggers it?
TT	TRUE	X>Z -> TRUE	WHILE loop runs (lines 10-14)	Input = 22
TF	TRUE	X>Z -> FALSE	ELSE branch runs (line 16)	NOT POSSIBLE in this program
FT	FALSE	Y>Z -> TRUE	DO-WHILE loop runs (lines 20-24)	NOT POSSIBLE in this program
FF	FALSE	Y>Z -> FALSE	ELSE branch runs (line 26)	Input = -22

Why are TF and FT impossible?

For input 22: X=1298 is the LARGEST number (bigger than both Y and Z). So if X>Y is TRUE, then X>Z is also TRUE. You can never have X>Y true but X>Z false. Similarly for input -22: X is the SMALLEST, so both IFs are false. The math of the formulas makes TF and FT unreachable!

Decision Coverage Results:

Test Input(s)	Path(s) Covered	Decision Coverage	Explanation
22 only	TT only	$1/4 = 25\%$	Only 1 of 4 paths tested
-22 only	FF only	$1/4 = 25\%$	Only 1 of 4 paths tested

22 AND -22	TT + FF	2/4 = 50%	Best possible! TF and FT unreachable
------------	---------	-----------	--------------------------------------

(e) 100% Statement Coverage

100% statement coverage means EVERY line in the program executes at least once. To check if it is possible, you need to see if there is any input that reaches ALL branches.

Answer: 100% Statement Coverage is NOT possible in this program.

Maximum you can achieve = 75% (using both 22 and -22 combined).

Why? Because lines 15,16 (ELSE of inner IF when outer is TRUE) and lines 20-24 (DO-WHILE block) can never execute for any roll number input. The math of the formulas always makes certain paths unreachable.

(f) 100% Decision Coverage

Answer: 100% Decision Coverage is also NOT possible in this program.

Maximum = 50% (2 out of 4 paths) using inputs 22 and -22.

The TF and FT paths can never be triggered by any input because of how X, Y, Z are mathematically related to each other through the roll number.

(g) Decision Coverage Using Roll No (022)

Last 3 digits of roll number = **022**. So the test input is **22**.

→ Input 22 -> executes the TT path (both IFs true)

→ 1 out of 4 decision paths is covered

Decision Coverage with input 022 = $1 / 4 \times 100 = 25\%$

(h) Statement Coverage Using Roll No (022)

Using **022** as input, statement coverage = **57.14%**

Lines that EXECUTE (16 lines):

- 1, 2, 3, 4, 5, 6, 7
- 8 (outer IF)
- 9 (inner IF)
- 10, 11, 12, 13
- 14, 28

Lines that DO NOT execute (12 lines):

- 15, 16, 17 (ELSE of inner IF)
- 18 (ELSE of outer IF)
- 19 (second inner IF)
- 20, 21, 22, 23
- 24, 25, 26, 27

Statement Coverage with input 022 = $16 / 28 \times 100 = 57.14\%$

() Final Cheat Sheet - Everything at a Glance

Topic	Key Concept	Formula / Answer
CFG	Only decision statements (IF, WHILE, ELSE, END IF...). Drop everything else.	13 decision nodes remain after removing lines 1-7 and non-decisions
HFG	All executable statements. Group consecutive non-decision ones.	9 groups. Decision nodes stay individual.
Statement Coverage	How many lines execute for a given input?	$(\text{Lines run} / 28) \times 100$
Decision Coverage	How many branch paths are tested?	$(\text{Paths covered} / 4) \times 100$
Input 22	Takes TRUE-TRUE path	Stmt: 57.14% Decision: 25%
Input -22	Takes FALSE-FALSE path	Stmt: 50% Decision: 25%
22 + (-22)	Best combination possible	Stmt: 75%* Decision: 50%
100% Coverage	NOT achievable in this program	TF and FT paths are mathematically unreachable
Roll No 022	Use 22 as test input	Stmt: 57.14% Decision: 25% (TT path)

Memory trick for T/F in diagrams:

T = LEFT side of the diamond. F = RIGHT side of the diamond. ALWAYS.

Pre-test loop (WHILE): condition box at TOP, loops back UP on True.

Post-test loop (DO-WHILE): DO box at TOP, body runs, condition box at BOTTOM, loops back UP on True.